ARQUITETURA DE COMPUTADORES · AULA 04

# MASM, NASM E MIPS

*Sintaxes diferentes representam operações sobre registradores e memória.*

MODELOS

# MONTADORES E CONJUNTOS DE INSTRUÇÕES TÊM CONVENÇÕES PRÓPRIAS

### x86 · CISC

MASM e NASM usam instruções com dois operandos na aritmética apresentada; o destino também é uma fonte.

### MIPS · RISC

Instruções aritméticas como add usam destino e duas fontes explícitas.

SINTAXE

# UMA MESMA INTENÇÃO PODE USAR NOMES E FORMATOS DIFERENTES

| OPERAÇÃO | MASM · X86 32 BITS | NASM · X86-64 | MIPS · 32 BITS |
|---|---|---|---|
| Carregar constante | `MOV EAX, 10` | `mov rax, 10` | `li $t0, 10` |
| Somar registradores | `ADD EAX, EBX` | `add rax, rbx` | `add $t0, $t1, $t2` |
| Subtrair registradores | `SUB EAX, EBX` | `sub rax, rbx` | `sub $t0, $t1, $t2` |
| Seção de dados | `SEGMENT` | `section .data` | `.data` |

OPERANDOS

# X86 USA DESTINO E FONTE; MIPS NOMEIA DUAS FONTES

### X86 · DOIS OPERANDOS

`add eax, ebx` atualiza EAX com EAX + EBX.

O primeiro operando é destino e também participa do cálculo.

### MIPS · TRÊS OPERANDOS

`add $t0, $t1, $t2` atualiza $t0 com $t1 + $t2.

Destino e fontes aparecem separadamente.

# O OPERANDO PODE VIR DA INSTRUÇÃO OU DE UM REGISTRADOR

### Imediato

Uma constante escrita na própria instrução, como `mov rax, 10`.

### Registrador

Um valor já armazenado em registrador, como a fonte de `add rax, rbx`.

MIPS MIPS usa `addi` para somar uma constante com sinal. Não existe uma instrução básica `subi`; subtrair 5 pode ser escrito como `addi $t0, $t1, -5`.

RASTREAMENTO

# A INSTRUÇÃO SUB ALTERA O DESTINO E PRESERVA A FONTE

```
mov rax, 80
mov rcx, 30
sub rax, rcx
```

| ETAPA | RAX | RCX |
|---|---|---|
| Inicial | — | — |
| Após primeiro `mov` | 80 | — |
| Após segundo `mov` | 80 | 30 |
| Após `sub rax, rcx` | 50 | 30 |

REUTILIZAÇÃO

# EXPRESSÕES MAIORES VIRAM UMA SEQUÊNCIA DE OPERAÇÕES SIMPLES

**CARREGAR**
colocar operandos em registradores

→

**OPERAR**
somar ou subtrair

→

**ACUMULAR**
reutilizar resultados intermediários

→

**GUARDAR**
manter o resultado final

Acompanhar o estado após cada instrução revela onde o resultado parcial fica.

DESTINO E FONTES

# LEIA A ORDEM DOS OPERANDOS ANTES DE COMPARAR INSTRUÇÕES

| FORMA | LEITURA |
| --- | --- |
| x86: `sub eax, ebx` | EAX recebe EAX − EBX |
| MIPS: `sub $t0, $t1, $t2` | $t0 recebe $t1 − $t2 |
| MIPS: `addi $t0, $t1, -5` | $t0 recebe $t1 − 5 |

**CUIDADO** Copiar apenas o nome da operação sem ajustar a ordem dos operandos pode mudar o resultado.

SÍNTESE

# A LÓGICA É COMUM; A SINTAXE E O FORMATO DOS OPERANDOS VARIAM

- MASM e NASM usam convenções x86 de dois operandos na aritmética mostrada.
- MIPS separa destino e duas fontes em operações como add.
- Imediatos são constantes; operandos em registradores vêm de valores armazenados.
- Rastrear registradores após cada instrução permite conferir o cálculo.

REFERÊNCIAS

# LEITURAS DA AULA

Tanenbaum, A. S. *Organização Estruturada de Computadores*. Patterson, D. A.; Hennessy, J. L. *Organização e Projeto de Computadores*.